

BÀI THỰC HÀNH OBSERVER PATTERN

- ✓ Backend: ASP.NET Core Web API
- ✓ Frontend: React (gọi API, trigger event)
- ✓ Có Observer Pattern đúng chuẩn GoF
- ✓ Có nhiều Observer (Email, SMS, Logging)

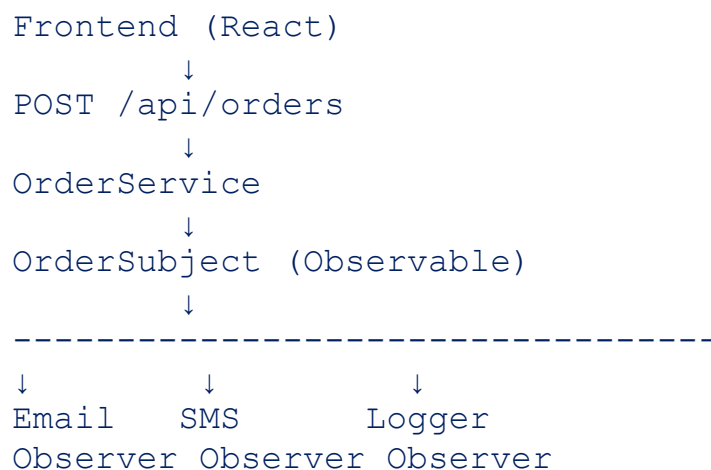
BÀI TOÁN

Hệ thống E-commerce khi **đơn hàng được tạo thành công** sẽ tự động:

-  Gửi Email cho khách hàng
-  Gửi SMS
-  Ghi log hệ thống

 Các hành động này **không hard-code trong OrderService** mà sử dụng **Observer Pattern**

KIẾN TRÚC HỆ THỐNG



 Khi Order được tạo → Notify tất cả observers

BACKEND – ASP.NET Core 8 Web API

1 Tạo project

```
dotnet new webapi -n ObserverDemo
cd ObserverDemo
```

📁 Cấu trúc thư mục

```
ObserverDemo
├── Controllers
│   └── OrdersController.cs
├── Services
│   └── OrderService.cs
├── Observers
│   ├── IObserver.cs
│   ├── EmailObserver.cs
│   ├── SmsObserver.cs
│   └── LoggingObserver.cs
├── Subjects
│   ├── ISubject.cs
│   └── OrderSubject.cs
├── Models
│   ├── Order.cs
│   └── OrderResult.cs
└── Program.cs
```

🌀 1. OBSERVER INTERFACE

Observers/IObserver.cs

```
using ObserverDemo.Models;

namespace ObserverDemo.Observers;

public interface IObserver
{
    Task Update(Order order);
}
```

🌀 2. SUBJECT (OBSERVABLE)

Subjects/ISubject.cs

```
using ObserverDemo.Observers;
using ObserverDemo.Models;

namespace ObserverDemo.Subjects;

public interface ISubject
{
    void Attach(IObserver observer);
    void Detach(IObserver observer);
    Task Notify(Order order);
}
```

Subjects/OrderSubject.cs

```
using ObserverDemo.Models;
using ObserverDemo.Observers;

namespace ObserverDemo.Subjects;

public class OrderSubject : ISubject
{
    private readonly List<IObserver> _observers = new();

    public void Attach(IObserver observer)
    {
        _observers.Add(observer);
    }

    public void Detach(IObserver observer)
    {
        _observers.Remove(observer);
    }

    public async Task Notify(Order order)
    {
        foreach (var observer in _observers)
        {
            await observer.Update(order);
        }
    }
}
```

✿ 3. CONCRETE OBSERVERS

EmailObserver.cs

```
using ObserverDemo.Models;

namespace ObserverDemo.Observers;

public class EmailObserver : IObserver
{
    public async Task Update(Order order)
    {
        await Task.Delay(200);
        Console.WriteLine($"✉ Email sent for Order
{order.Id}");
    }
}
```

SmsObserver.cs

```
using ObserverDemo.Models;

namespace ObserverDemo.Observers;

public class SmsObserver : IObserver
{
    public async Task Update(Order order)
    {
        await Task.Delay(200);
        Console.WriteLine($"📱 SMS sent for Order
{order.Id}");
    }
}
```

LoggingObserver.cs

```
using ObserverDemo.Models;

namespace ObserverDemo.Observers;
```

```
public class LoggingObserver : IObservable
{
    public async Task Update(Order order)
    {
        await Task.Delay(100);
        Console.WriteLine($"📝 Order {order.Id} logged");
    }
}
```

🔗 4. MODEL

Models/Order.cs

```
namespace ObserverDemo.Models;

public class Order
{
    public int Id { get; set; }
    public decimal Amount { get; set; }
}
```

Models/OrderResult.cs

```
namespace ObserverDemo.Models;

public class OrderResult
{
    public string Message { get; set; }
}
```

🔗 5. ORDER SERVICE

Services/OrderService.cs

```
using ObserverDemo.Models;
using ObserverDemo.Subjects;

namespace ObserverDemo.Services;
```

```

public class OrderService
{
    private readonly OrderSubject _subject;

    public OrderService(OrderSubject subject)
    {
        _subject = subject;
    }

    public async Task<OrderResult> CreateOrder(Order order)
    {
        // Giả lập lưu DB
        await Task.Delay(300);

        // Trigger event
        await _subject.Notify(order);

        return new OrderResult
        {
            Message = $"Order {order.Id} created
successfully"
        };
    }
}

```

✿ 6. CONTROLLER

Controllers/OrdersController.cs

```

using Microsoft.AspNetCore.Mvc;
using ObserverDemo.Models;
using ObserverDemo.Services;

namespace ObserverDemo.Controllers;

[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly OrderService _orderService;

    public OrdersController(OrderService orderService)

```

```

    {
        _orderService = orderService;
    }

    [HttpPost]
    public async Task<IActionResult> Create(Order order)
    {
        var result = await
        _orderService.CreateOrder(order);
        return Ok(result);
    }
}

```

7. ĐĂNG KÝ DI + OBSERVERS

Program.cs

```

using ObserverDemo.Observers;
using ObserverDemo.Services;
using ObserverDemo.Subjects;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

// Subject
builder.Services.AddSingleton<OrderSubject>();

// Observers
builder.Services.AddSingleton<IObserver, EmailObserver>();
builder.Services.AddSingleton<IObserver, SmsObserver>();
builder.Services.AddSingleton<IObserver,
LoggingObserver>();

// Service
builder.Services.AddTransient<OrderService>();

// Swagger
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// CORS

```

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowReact",
        policy =>
        {
            policy.WithOrigins("http://localhost:3000")
                .AllowAnyHeader()
                .AllowAnyMethod();
        });
});

var app = builder.Build();

// Gắn observers vào subject
var subject =
app.Services.GetRequiredService<OrderSubject>();
var observers = app.Services.GetServices<IObserver>();

foreach (var observer in observers)
{
    subject.Attach(observer);
}

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseCors("AllowReact");
app.MapControllers();
app.Run();
```

CHẠY BACKEND

dotnet run

Swagger:

<https://localhost:xxxx/swagger>

Test API:

POST /api/orders

Body:

```
{
  "id": 1,
  "amount": 200
}
```

FRONTEND – REACT

Tạo project

```
npx create-react-app observer-ui
cd observer-ui
npm install axios
```

src/App.js

```
import React, { useState } from "react";
import axios from "axios";

function App() {
  const [amount, setAmount] = useState(100);
  const [result, setResult] = useState("");

  const createOrder = async () => {
    const response = await axios.post(
      "https://localhost:xxxx/api/orders",
      {
        id: Math.floor(Math.random() * 1000),
        amount: amount,
      }
    );

    setResult(response.data.message);
  };

  return (
    <div style={{ padding: 40 }}>
      <h2>Create Order</h2>
    </div>
  );
}
```

```
    <label>Amount:</label>
    <input
      type="number"
      value={amount}
      onChange={ (e) => setAmount(e.target.value) }
    />

    <br /><br />

    <button onClick={createOrder}>Create Order</button>

    <h3>{result}</h3>
  </div>
);
}

export default App;
```

▶ CHẠY FRONTEND

```
npm start
```

💡 LUỒNG XỬ LÝ OBSERVER

Khi tạo Order:

1. React gửi request
2. Controller gọi `OrderService.CreateOrder()`
3. Service gọi:

```
_subject.Notify(order);
```

4. Subject loop qua observers:

- EmailObserver → gửi email
 - SmsObserver → gửi SMS
 - LoggingObserver → ghi log
-

💡 SO SÁNH VỚI CODE KHÔNG DÙNG OBSERVER

✗ Cách sai (tight coupling):

```
SendEmail();  
SendSms();  
WriteLog();
```

✓ Cách đúng (Observer):

```
_subject.Notify(order);
```

🔗 Để mở rộng:

- Thêm PushNotificationObserver
- Thêm KafkaEventObserver
- Không sửa OrderService

🚀 BÀI TẬP MỞ RỘNG

1. Thêm PushNotificationObserver
2. Thêm DiscountObserver (áp dụng coupon sau khi order)
3. Gửi message qua RabbitMQ / Kafka
4. Log ra file thay vì Console